A PRACTICAL TEAM GUIDE

CN SDN Project

Priority-based log forwarding using UDP, Mininet, Open vSwitch and Ryu

Goal
Send application logs from multiple clients, forward them through an SDN-controlled network, and measure whether priority treatment helps CRITICAL traffic while INFO traffic continues to make progress.

How the parts fit together

Part	Main job	Owner
1. Application traffic	Clients send JSON logs; server receives and measures them.	Person 1
2. Network and tests	Mininet creates hosts/switches and repeatable congestion experiments.	Person 2
3. SDN control	Ryu installs rules; OVS queues apply real traffic scheduling.	Person 3
Integration	Agree on shared fields, addresses, priority mapping and result format.	Everyone

Agree on these once before integration

- JSON fields: client_id, sequence, level, priority, timestamp. The message text is optional.
- Priority mapping: INFO = 1, WARNING = 2, ERROR = 2, CRITICAL = 3 (change only if the team agrees).
- Choose one network-visible classification method: dedicated UDP destination ports per class OR DSCP markings.
- Keep server address/port, switch name, real switch ports, queue IDs and queue settings consistent across parts.

Use the shared config for common settings. Values such as switch ports and queue IDs are examples until confirmed against the actual Mininet/OVS topology.

Part 1 — Person 1

Purpose: prove the application traffic works locally on your Mac before adding Mininet or Ryu.

Files to maintain

File	Only what it needs to do
part1_app/client.py	Send UDP JSON logs for a fixed duration at a configured rate. Use a client ID and log level; increase sequence for each log.
part1_app/server.py	Receive UDP datagrams, parse JSON, check the required fields, print valid logs, count them and calculate received throughput.
part1_app/load_test.py	Start the server and at least three client processes, collect their sent counts, read the server summary and save one JSON result.
config/project_config.json	Shared server IP/port, log priorities, test duration, client count/rate, and the agreed switch/queue settings.
results/	Store load_test_summary.json and server_output.txt for evidence.

Message format

```
{
  "client_id": "H1",
  "sequence": 1,
  "level": "CRITICAL",
  "priority": 3,
  "timestamp": "2026-10-13T12:00:00+00:00"
}
```

With the current client.py

The client reads config/project_config.json, takes its client ID and level from command-line arguments, sends JSON over UDP, and prints its final sent count. Run commands from the project root because the config path is relative.

```
python3 part1_app/load_test.py

# The load test launches clients like this internally:
python3 part1_app/client.py H1 INFO
```

Person 1 is done when

- At least three client IDs appear at the receiver, with their log levels.
- The summary includes total sent, valid received, estimated loss and received logs/second.
- The machine-readable result is saved under results/. Record the duration, client count, rate and priority mix.
- The local baseline runs without Mininet or Ryu.

Metric definitions: Sent = datagrams the clients attempted to send. Received = valid log datagrams counted by the server. Estimated loss = sent − received for the same test window; it is an estimate because UDP can lose packets and timing windows matter. Throughput = valid received logs / measured receiving interval.

Current-code note: project_config.json contains a trials setting, but the current local load_test.py performs one run. Repeated trials for the experiments belong in Person 2’s experiment runner unless the team later decides to add repeats to the local test.

Part 2 — Person 2

Purpose: run the same client/server application through a virtual SDN network and create repeatable congestion.

Files to create

File / output	Required work
part2_mininet/topology.py	Create hosts H1, H2, H3, switch s1 and receiver HR. Set links, document IP addresses and record the actual switch port numbers.
part2_mininet/run_experiment.sh	Start/stop applications and traffic, run experiments A–D, and save a log plus JSON/CSV summary for each run.
results/	Keep per-run summaries and console logs, plus a small note of the test date, topology, offered load and queue settings.

Four experiments

Run	Setup	Question answered
A — Low load	A few clients or low send rate; no priority needed.	Does basic connectivity and log delivery work?
B — Congested baseline	High competing traffic; priority is disabled/default.	What is the throughput/loss under congestion without priority?
C — Priority	Same offered load and duration as B; priority rules and queues enabled.	Does CRITICAL improve compared with B?
D — Fairness stress	Sustained CRITICAL plus INFO traffic with the fairness policy enabled.	Does INFO keep making progress under heavy CRITICAL load?

Rules for fair tests

- Run all four experiments. Do not stop after B and C.
- Repeat B and C at least three times with the same traffic load and duration. Change the priority policy, not the offered load.
- Record client count, offered rate, duration, priority mix, queue settings, trial number and test time.
- Run the existing Person 1 scripts inside Mininet; do not rewrite the application just because it is running on virtual hosts.
- Use the receiver host’s reachable Mininet IP. Inside a Mininet host, 127.0.0.1 means that same host, not HR.

Person 2 is done when the topology works, A succeeds, congestion is measurable, A–D run in order, and every run saves enough settings and output to reproduce the test.

Part 3 — Person 3

Purpose: make the SDN switch classify traffic and actually schedule it through configured queues.

Files to create

File	Required work
part3_controller/controller.py	A Ryu app that connects to the switch, installs forwarding rules and selects queues using actions supported by the chosen OVS/OpenFlow setup.
part3_controller/setup_queues.sh	Configure OVS QoS and queues on the correct egress port; set and document queue IDs and rates/weights.
Verification notes	Record the commands and evidence showing installed flows, configured queues, traffic counters and packets arriving at the server.

Choose and document one classifier

Option	What the team must do
Dedicated UDP ports	Give each class an agreed destination port. Make the receiver/topology listen on those ports or redirect traffic appropriately.
DSCP markings	Mark traffic at the sender and use switch rules that match the selected DSCP value. Verify support in the installed OVS/OpenFlow/Ryu versions.

Critical technical point

An ordinary OpenFlow rule does not read arbitrary JSON text such as "level": "CRITICAL" inside a UDP payload. The class must be represented by a network-visible field such as the agreed UDP destination port or DSCP. Selecting a queue in a rule is not enough: the queue must be configured on the relevant egress port and actually used.

Fairness is required

- Do not rely on strict priority alone if it can starve INFO.
- Use weighted/fair scheduling or appropriate rate limits so lower-priority traffic continues to receive service.
- Show that the controller connects, expected rules exist, packets match the intended class, queue counters change, CRITICAL improves in C versus B, and INFO continues during D.

Person 3 is done when rules, queue configuration and traffic behavior are all verified with evidence - not merely when the controller starts without an error.

Coding style for everyone (and AI)

Write code in the same spirit as the existing beginner-friendly socket program: direct, readable and only as complex as the requirements demand.

- Question every line. Include it only if it meets a requirement or makes the code easier to understand.
- Prefer straightforward imports, uppercase constants, clear variable names, simple loops and small functions only when useful.
- Avoid unnecessary classes, design patterns, layers of helper functions, excessive try/except blocks, and complicated parsing.
- Use Python's standard library where possible. Add only packages required by Mininet/Ryu or the chosen tools.
- Do not add ACK/retry logic, a separate log generator, a logging framework or extra features unless the team has a real requirement for them.
- Do not silently change the JSON fields, log levels, priority numbers, classifier, IP/port or queue mapping.
- When editing code, provide the entire updated file, not only a snippet, and include the exact command to run it.
- Test one small case first, then three or more clients, then integrate with Mininet and Ryu.

Suggested repository layout

```
cn-sdn-project/
├── part1_app/
│   ├── client.py
│   ├── server.py
│   └── load_test.py
├── part2_mininet/
│   ├── topology.py
│   └── run_experiment.sh
├── part3_controller/
│   ├── controller.py
│   └── setup_queues.sh
├── config/
│   └── project_config.json
├── results/
└── README.md
```

Final demo checklist

- Show the shared config and explain the priority/classification mapping.
- Show at least three local clients and the Person 1 sent/received/throughput summary.
- Show Mininet topology and successful Run A.
- Show Ryu flow rules and verify OVS queues/counters.
- Show repeated, matched B/C runs and Run D with INFO progress under sustained CRITICAL load.
- Present throughput, estimated loss, and latency only if measured, together with test conditions.

> The aim is not the fewest lines at any cost. It is the fewest clear lines that still satisfy the project requirements.